FINAL PROJECT

TCP6114 – COMPUTER PROGRAMMING

TRIMESTER OCT/NOV, 2024/2025 (Term 2430)

Lab Section: 1H

Group: G9

Title: Tech Product Management System

Submission Date: 7th February 2025

NO	STUDENT ID	STUDENT NAME	MAJOR
1.	241UT240K7	Cheong Kai Jin	AI
2.	241UT240W1	Kang Xuan Xin	AI

Table of Contents

Task Distribution 2

Project Evaluation Form 3

Introduction 4

Problem Statements 5

Objectives 6

Program Scope 7

Detailed Description 9

Screenshot of the program 22

Source Code 33

Bibliography 40

Appendix 41

Task Distribution

No.	Student ID	Student Name	Tasks Completed
1.	241UT240K7	Cheong Kai Jin	Developing and debugging the program. Writing a detailed description of the program. Prepare chart and diagrams. Preparing the presentation.
2.	241UT240W1	Kang Xuan Xin	Writing the Introduction, Problem Statement, Objectives, Program Scope and Screenshot of the Program. Assist in developing, debugging and testing the program. Preparing the presentation.

Project Evaluation Form

Part 1 – Program and Documentation

Assessment Criteria	Marks
1. Content and Interface (15%) Content relevance, interface appropriateness, content organization and presentation	
2. Functionality (15%) The program must contain at least 3 of the functionalities (add, edit, delete, view and search)	
3. Object-oriented Implementation (15%) Apply the OOP concept and use classes to solve the problem	
4. **Completeness (15%)** Completeness of the C++ program and free from bugs and errors	
5. Documentation (20%) Quality of the report - Ability to describe the project and system developed and provide the necessary information. The quality of document formatting.	
Sub-Total (80%) (A)	

Part 2 – Level of Understanding

STUDENT ID	STUDENT NAME	Level of understanding (20%) (B)	TOTAL MARKS (100%) (A+B)
241UT240K7	**Cheong Kai Jin**		
241UT240W1	**Kang Xuan Xin**		

Remark by Evaluator (if any): *E.g. Plagiarism with other groups/online sources/AI-generated code, 0 mark given*	

Introduction

Introduce your topic. Tell the reader what the topic is about. Describe the main part and explain the key points of your program.

This Tech Product Management System is a console-based UI product design that is efficient in managing product information. File handling is used for storing product IDs and attributes in text files for easy and simple handling [1]. Users can perform some operations, such as adding, listing all products, viewing certain product's information, editing product's information, or deleting on products. Then there can be several attributes with each product, for example, specifications; CPU, Battery, etc. stored in key-value pairs.

The program functions on object-oriented programming (OOP) principles [2], working with two core classes, which is ***Idlist*** (managing individual unique product IDs) and ***Product*** (managing product details). From [3], the ***Idlist*** class manages a sorted alphabetical list of product IDs inside a file text that can be used for easy validation and deletion of duplicates. Besides, the ***Product*** class is responsible for managing attributes in distinct files, showing the organisation of data in a structured format.

Problem Statements

State the issues/problems to be addressed or conditions to be improved that lead to the development of the program.

According to research from [4], the handling of product records manually jeopardises the efficiency, accuracies, and scalability. One primary concern is manual records management, requiring substantial human power for product information maintenance and modification. Staffs must input, arrange, and locate product details, which reduces time they should be used for more crucial activities.

Besides, one of the major concerns is very high probability of human error in data entry and retrieval. Errors such as streamlined records, wrong product information, and displaced information, end up causing operational inefficiencies and financial losses [5].

In addition, record keeping physically on paper is occupying a lot of space and is a resource-constrained task. Businesses often require storage space for paper files, which not only take up space and make it harder to find documents, but they also increase risks of losing a valuable information. In consequence, document searches get harder and take longer, and office spaces tend to become messy as companies grow [6].

Lastly, manual record-keeping is insecure and does not allow for real-time access to information. For instance, finding a specific record in the mass of files is a time-consuming event for us, and paper files can get lost, stolen, or damaged. Thus, unlike digital solutions, paper records prove inefficient for storage purposes especially for long-term use [7].

Objectives

State the objectives of what you want to achieve with your topic.

The main objective of the project is to simplify data maintenance for products, in the shortest possible time with the highest efficiency. Most companies often have a hard time keeping product details preserved if the current method for doing so is manual. Thus, our system looks forward to helping organically save as well as organise information about products within the company. In doing so, record keeping will be much less difficult [7].

Indeed, another objective is to reduce management costs. The costs of storage, printing, and staff time are a requisite for manual data entry and paper records. By going the path of the digital system, businesses can cut costs and improve accuracy, leading to better productivity and savings [8].

Furthermore, another objective of this project is to come up with user-friendly and robust tool. The idea is to come up with a system that is simple enough for a person even with non-technical skills to understand. We have ensured consistency in designing our system Interaction to let our user have a smooth experience in using our system [9].

Lastly, the system should also improve data security and enhance availability. Store product records digitally help protect important information from being lost, damaged, or accessed without proper authorisation. In comparison with written data, a digital record can be saved and viewed from anywhere, allowing companies to secure their data more efficiently [10].

To sum up, promoting these objectives shall allow the system to facilitate improved efficiencies, lowered costs, and a better and secure centralised system for managing product information.

Program Scope

Describe the boundaries of the entire program, e.g., features, functions, deliverables, constraints, etc.

Our program has been made on a basic file-oriented system to maintain information about technology products through a command-line interface. Here are its key boundaries and characteristics:

Features

Product Management	Create/Delete products with unique IDs
	List all product IDs in the system
	View detailed attributes of a specific product
Attribute Management	Add/Delete attributes (e.g., “CPU”, “Battery”) and their values for products
	Attributes are stored in a sorted format for readability
File-Based Storage	Stores product IDs in a master file “_idlist.txt”
	Each product’s attributes are saved in a separate text file (e.g., iphone_10.txt)
User Interaction	Simple Command Line Interface with commands like *add*, *view*, *edit*, *del*, *list*, and *help*
	Supports stacked inputs (e.g., add phone1 view phone2 in one line)

Deliverables

Core Program	Executable C++ OOP for managing products efficiently
	Creates and maintains a bin/ directory for data storage
Data Files	Automatically generates _idlist.txt and product-specific files (e.g., product123.txt)

Constraints

Input Limitations	Product IDs and attribute values cannot contain spaces (due to cin usage)
Storage	Relies on manual creation of the bin/ directory
	No encryption or security for stored data
Scalability	Designed for small-scale use (not optimised for large datasets)
	File I/O operations may slow performance with thousands of entries
GUI/Networking	No graphical interface or network access (pure Command Line Interface)
	No cloud sync or multi-user support
System Integration	Does not interface with databases such as SQL
	No backup/restore functionality

Detailed Description

Provide a comprehensive description of the program, covering, but not limited to:

- **A detailed explanation**
- **System design (use-case diagram, flow chart, pseudocode, etc)**
- **A draft interface design**
- **And etc**

Some prerequisite knowledge:

The term “idlist” and other similar alternatives will be used to indicate the Idlist class or the text file “_idlist.txt” depending on the context.

The term “product file” and other similar alternatives will be used to indicate the product’s data text file.

The term “attribute” is a superset of two instances: key and value. The term “attribute name” is sometimes used to indicate “key”.

```
#include <iostream>
#include <fstream> // file to library
using namespace std;


//consts

const string BIN_PATH = "bin/"; //directory to store idlist.txt and product files
```

Fig.0 headers and global vars.

Headers, Global variables and Declarations:

In our system, we will only use two built-in libraries, specifically “iostream” and “fstream”. “iostream” is used for input and output and “fstream” is used for file operations.

Since this is only a small program, we will not be using any libraries like <vector> or <filesystem>. This means we do not have access to prewritten containers that has dynamic sizes (eg. vectors, sets, maps). All functions and method will use the basic c++ features and the two headers above.

We also will not use any function declaration (or function prototyping) since this is a relatively small program.

Global constant string "BIN_PATH" is a string that determines which directory data is stored in. Data includes "_idlist.txt" (a text file including all product IDs) and product data text files.

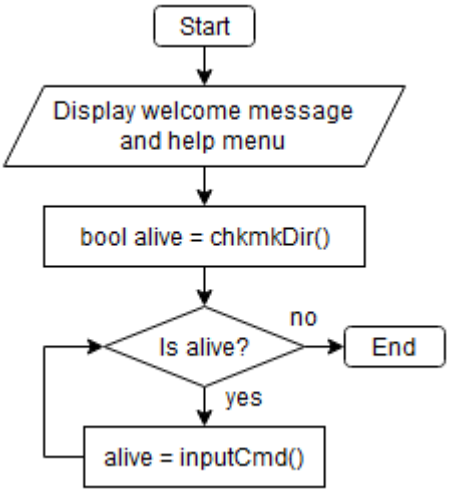


Fig.1 main flowchart

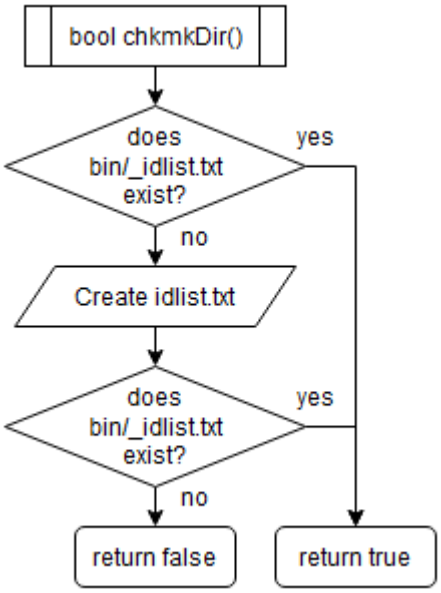


Fig.2 chkmkDir flowchart

Main: When the program starts, a welcome message is displayed and "cmdHelp()" is called to display the help menu.

Boolean "alive" determines if the program should continue running and process user input. Setting "alive" to TRUE keeps the program running while FALSE ends the program.

Boolean "alive" is initialized as TRUE when directory "bin/" exists and FALSE if otherwise. This is because the program requires the bin directory to store product data.

Function "chkmkDir()" checks if directory "bin/" or "_idlist.txt" is missing. If bin directory is missing, alive is set to false and program ends. If "_idlist.txt" is missing, a new one is created, and the program continues.

The function checks the existence of directory bin by first checking if "bin/_idlist.txt" exists, if it does not, it means either "idlist.txt" or the bin directory itself is missing. Then it attempts to create "_idlist.txt" inside the directory. If the text file still does not exist, it means the bin directory is missing.

After that, the program enters an **input loop**. If “alive” is TRUE, call “inputCmd()” and repeat. The loop ends when “alive” is FALSE.

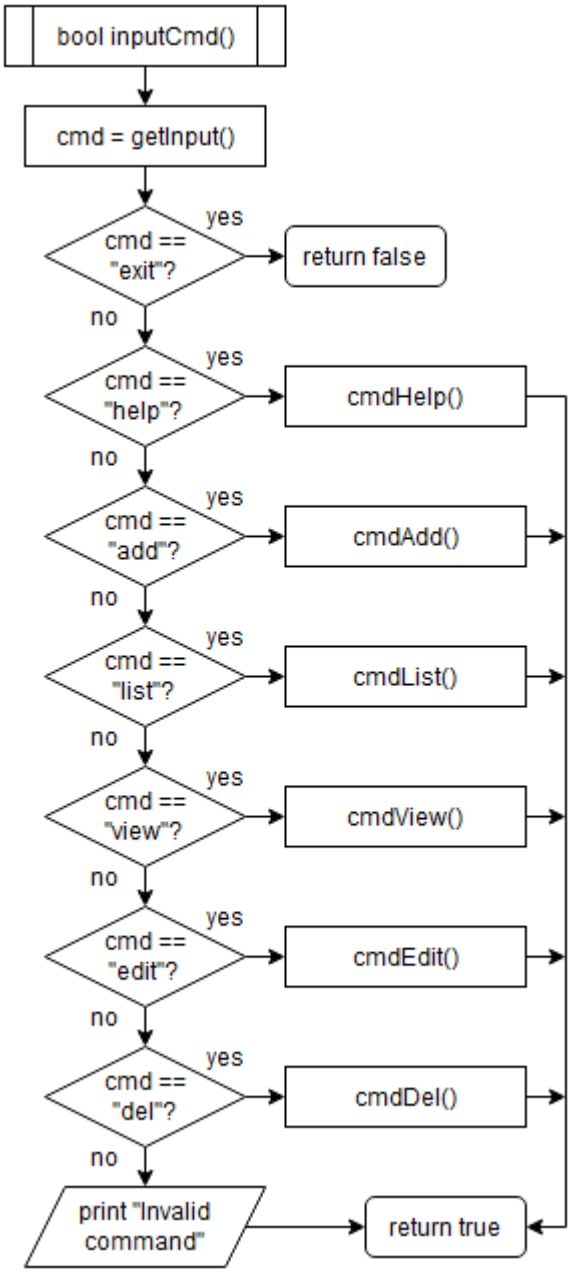


Fig.3 inputCmd flowchart

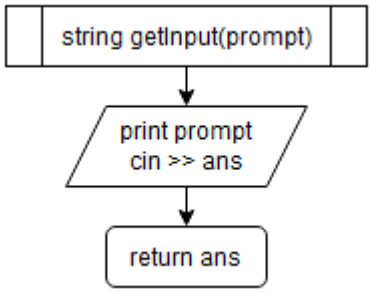


Fig.4 getInput flowchart

Function “inputCmd()” returns Boolean datatype which is similar to “chkmkDir()”, if user inputs “exit” command, it will return and set “alive” to FALSE, ending the program.

The program checks through every command and if none matches the input, “Invalid command” is printed, and the program continues. If user input matches one of the commands, the corresponding command function is called, and the program continues.

Function “getInput()” is a small helper function to improve code reusability. It prints the provided prompt message and returns back user input.

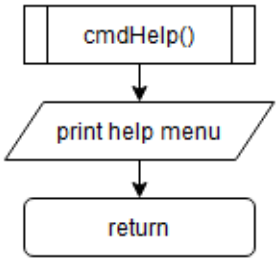


Fig.5 cmdHelp flowchart

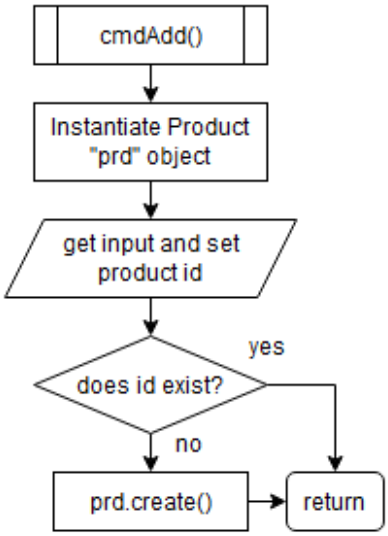


Fig.6 cmdAdd flowchart

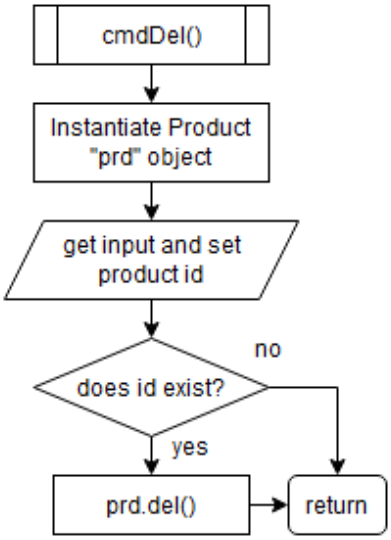


Fig.7 cmdDel flowchart

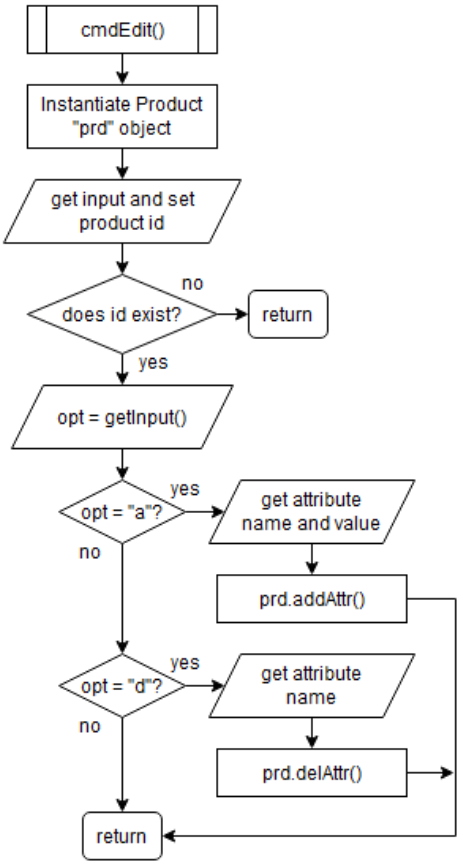


Fig.8 cmdEdit flowchart

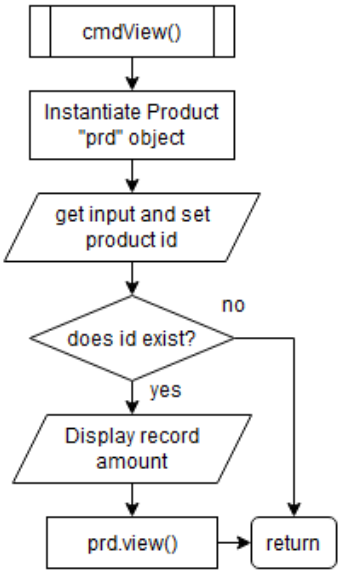


Fig.9 cmdView flowchart

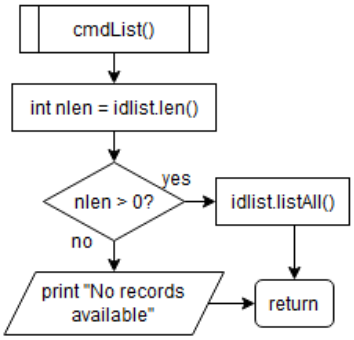


Fig.10 cmdList flowchart

Function "cmdHelp()" will print a pre-written help message/menu for users' reference.

Function "cmdAdd()", "cmdDel()", "cmdView()" and "cmdEdit()" all works similarly, that is instantiating an object named "prd" from the Product class and then operate by using the object's methods. The methods of Product and Idlist class will be introduced later.

Function "cmdAdd()" gets product id from user input and creates the product if the id is not taken.

Function "cmdDel()" gets product id from user input and deletes the product if it exists.

Function "cmdEdit()" gets product id from user input, if the id exists, it will then ask user to input option. Theres is two options: "a" meaning add or append attribute, "d" meaning delete attribute.

If ***option "a" (add) is inputted***, it will proceed to ask user to input attribute name and its value. Method "prd.addAttr(attr, val)" is then called, which writes the key-value pair into the product file. If ***option "d" (delete)*** is entered, the system will ask the user to input the attribute name for delete. If the attribute exists, method "prd.delAttr(attr)" is then called, and the system will delete the attribute and its corresponding value.

Function "cmdView()" gets product id from user input and if it exists, prints every attribute and its value that is stored in the product file.

Function "cmdList()" will first get the length of idlist. If the length is 0 (idlist is empty), it will print "No records available". If length is greater than 0, it will call "idlist.listAll()" method, which lists all identifiers that are stored in the "_idlist.txt" file.

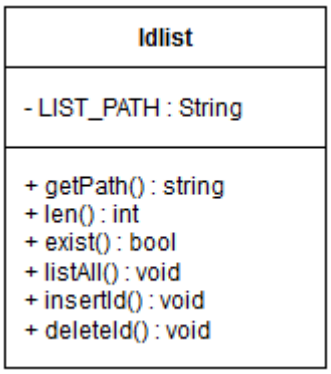


Fig.11 Idlist class UML diagram

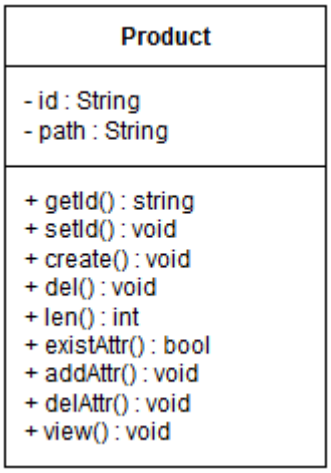


Fig.12 Product class UML diagram

Idlist is a collection of all existing product identifiers (IDs), the list is sorted alphabetically.

Idlist class is a class that represents idlist itself. All of its methods are to modify or query idlist.

Data member "LIST_PATH" is a private string constant that is predefined. It constains a path to "_idlist.txt" relative to the directory that the program runs in. In this case, its "bin/_idlist.txt".

Brief description for each method:

Method "getPath()" returns the path to idlist.

Method "len()" returns the length of the idlist (number of IDs).

Method "exist()" checks if an id already exists inside the idlist.

Method "listAll()" prints all IDs stored in idlist.

Method "insertId()" inserts a new id into idlist in alphabetical order.

Method "deleteId()" removes an existing id from idlist.

Product class is used to create, modify and delete product entries.

Data member "id" is a private string constant. It is used by methods to determine which product to operate on.

Data member "path" is a private string constant. It is a composite value that is the bin path combined with the id and ends with the ".txt" file ending.

Brief description for each method:

Method "getId()" returns the id string.

Method "setId()" assigns id and generates path.

Method "create()" creates a new empty text file using path. Id is the filename. (eg. "bin/iphone.txt")

Method "del()" deletes a file using path.

Method "len()" returns the length of the product file.

Method "existAttr()" checks if an attribute exists in the product file.

Method "addAttr()" inserts a new attribute and value into the product file in alphabetical order by attribute.

Method "delAttr()" finds and delete an existing attribute from the product file.

Method "view()" will print all attributes stored in the product file in a "key = value" fashion.

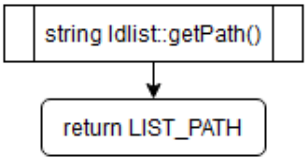
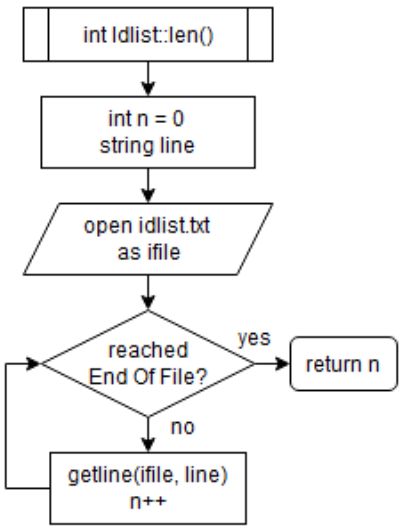
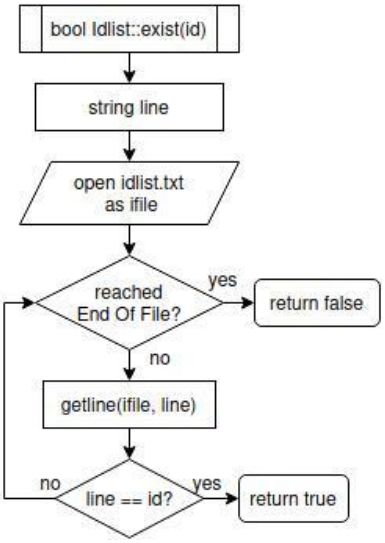


Fig.13 Idlist::getPath flowchart





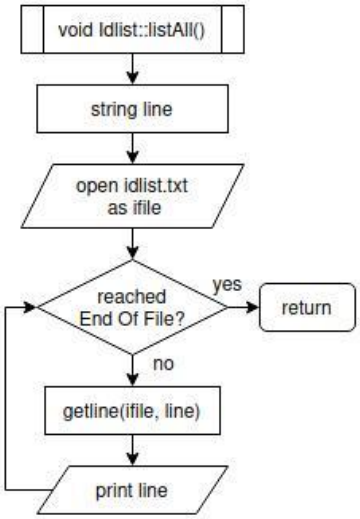


Fig.14 Idlist::len flowchart, Fig.15 Idlist::exist flowchart, Fig.16 Idlist::listAll flowchart.

Method "Idlist::getPath()" returns private data member "LIST_PATH" as string.

Method "Idlist::len()" initializes integer "n" and string "line", then opens "bin/_idlist.txt" as "ifile". It then enters a loop. If "ifile" reached End Of File (EOF), it will return int "n", If not, it will call "getline()" on ifile and increment "n".

Function "getline()" is a c++ built-in function that reads and removes a line from file stream and writes the line into a string.

Method "Idlist::exist()" initializes string "line" and opens opens "bin/_idlist.txt" as "ifile". It then enters a loop. If "ifile" reached EOF, it means it did not find the specified id inside idlist and returns FALSE, otherwise, call "getline(ifile, line)" and check if "line" is equal to parameter "id". If they match, return TRUE.

Method "Idlist::listAll()" initializes string "line" and opens idlist as ifile. It then iterates through every line in ifile and prints out the content until it reaches EOF.

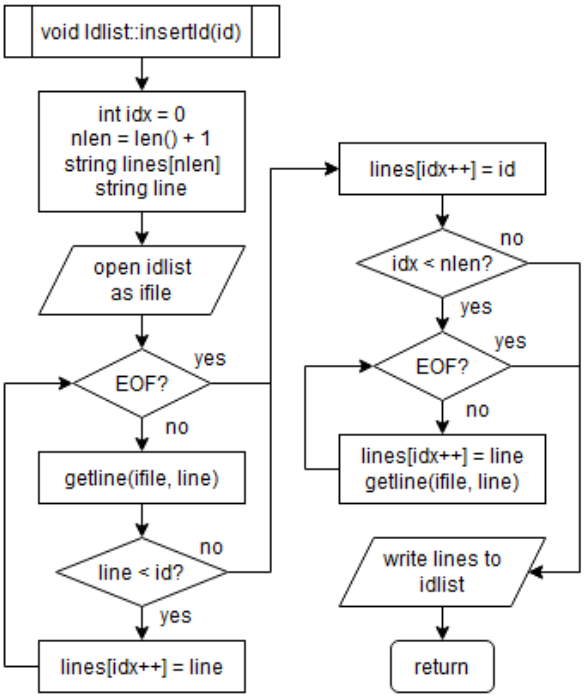


Fig.17 Idlist::insertId() flowchart

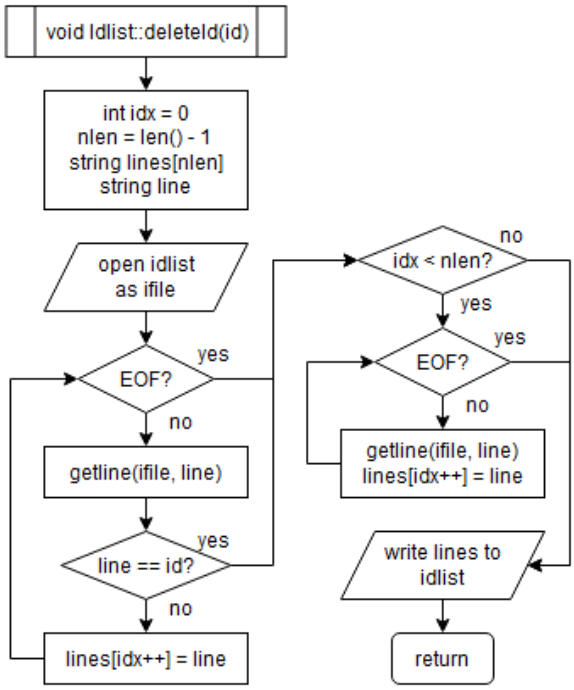


Fig.18 Idlist::deleteId() flowchart

Method "Idlist::insertId()" works by opening the "idlist.txt" as ifstream ifile, then checking which line is bigger than the provided id. When it checks each line, it also adds the line into an array called "lines". When it finds a line that is bigger than the provided id, it inserts the id before the line into array "lines", then adding the line that is bigger. It will proceed to add the remaining lines into array "lines" and finally write array "lines" into idlist. It functions like inserting an element into a sorted array. Integer "nlen" means "new length" and indicates the new length of the idlist. Inserting an id into idlist means length will be incremented by 1, thus len() + 1.

Method "Idlist::deleteId()" behaves similarly to "insertId()". But instead of finding a line smaller than id, it finds the line that matches the id. It will then discard the line (id) by not adding the id into the array "lines". Then it will proceed to add remaining lines and finally write the array into idlist.

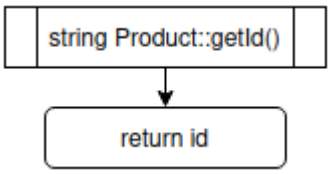


Fig.19 Product::getId flowchart

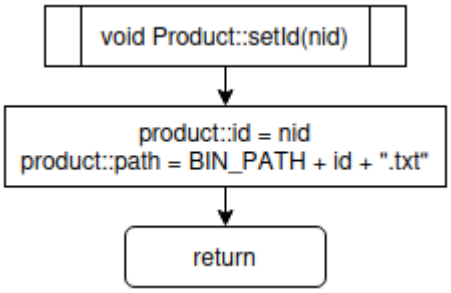
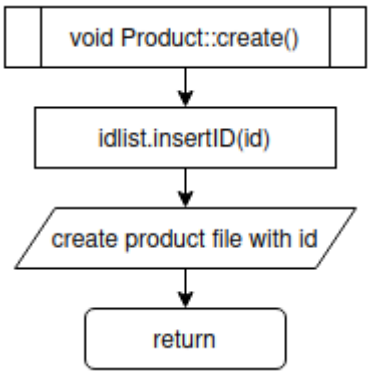


Fig.20 Product::setId flowchart

Method "Product::getId()" returns data member "id" as string.

Method "Product::setId()" assigns data member "id" with provided string parameter and generates a composite value named "path" which consists of BIN_PATH + id + ".txt".



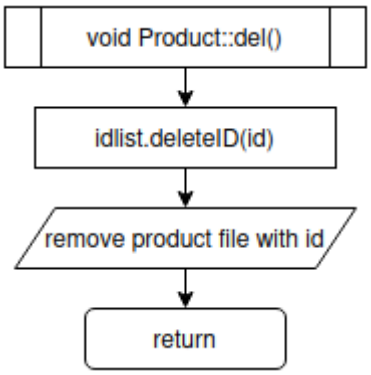
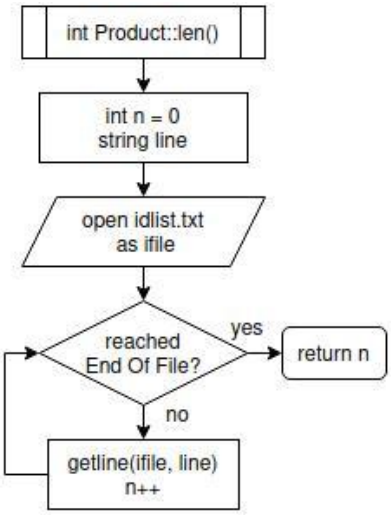
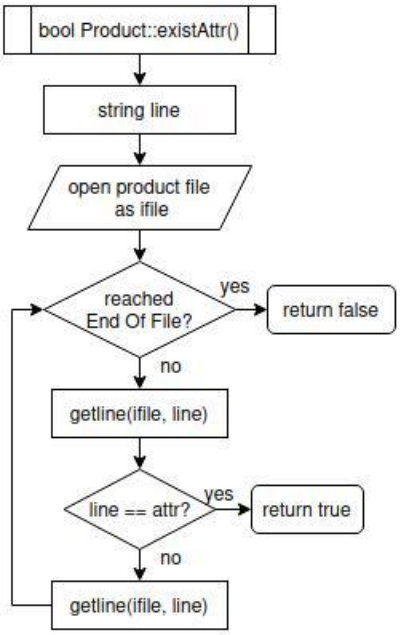


Fig.21 Product::create flowchart *Fig.22 Product::del flowchart*

Method "Product::create()" uses data member "id" to create a new text file located in BIN_PATH.

Method "Product::del()" deletes a text file with name that matches data member "id".





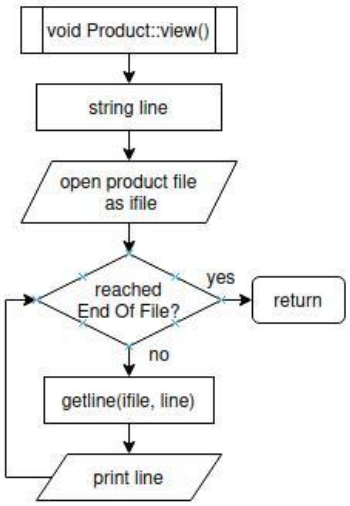


Fig.23 Product::len flowchart, Fig.24 Product::existAttr flowchart, Fig.25 Product::view flowchart.

Method "Product::len()" works similar to method "Idlist::len()", it iterates through the product file content and prints the length.

Method "Product::existAttr()" works similar to method "Idlist::exist()". But instead of iterating line by line, it skips one line in between iterations. This is because attributes are stored in two lines, the first (odd) line is the key, and second (even) line is the value.

Method "Product::view()" works similar to method "Idlist::listAll()". But instead of printing every line in order, it prints the key-value pair in "key = value" format. We already know that every odd line is a key and every even line is a value, so we can find an attribute every two lines.

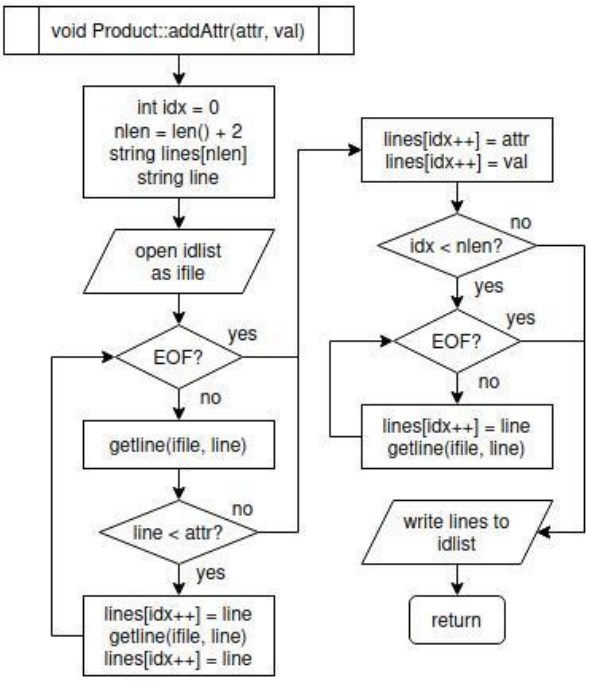


Fig.26 Product::addAttr flowchart

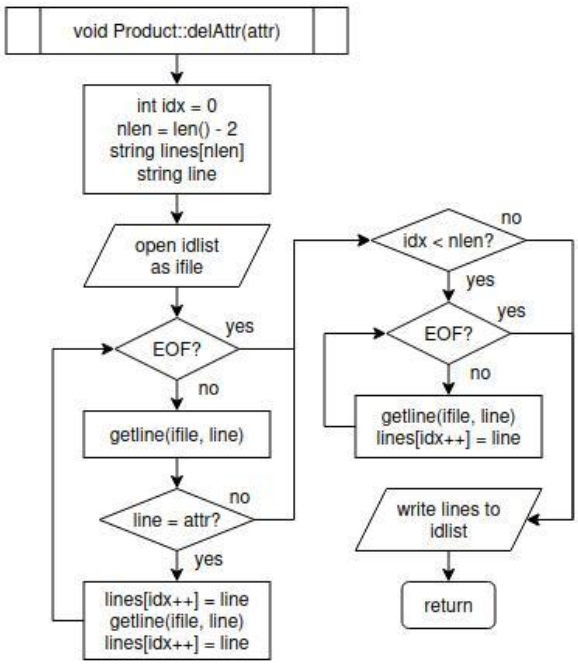
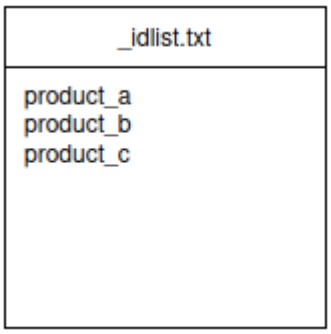


Fig.27 Product::delAttr flowchart

Method "Product::addAttr()" behaves like method "Idlist::insertId()". But instead of checking every line, it checks every two lines for attributes. It also inserts two elements into the array "lines" which is the key and value.

Method "Product::delAttr()" behaves like method "Idlist::deleteId()". And same as every other Product class method, it checks for attribute every two lines. When is finds the matching attribute name (key), it will discard the attribute by skipping the next two lines.

Then it will proceed to add the remaining lines and finally write array “lines” into the product file.



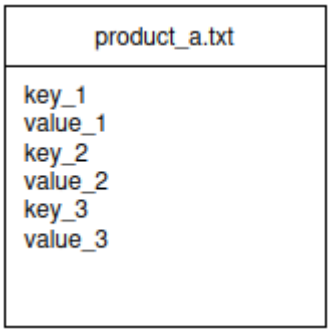


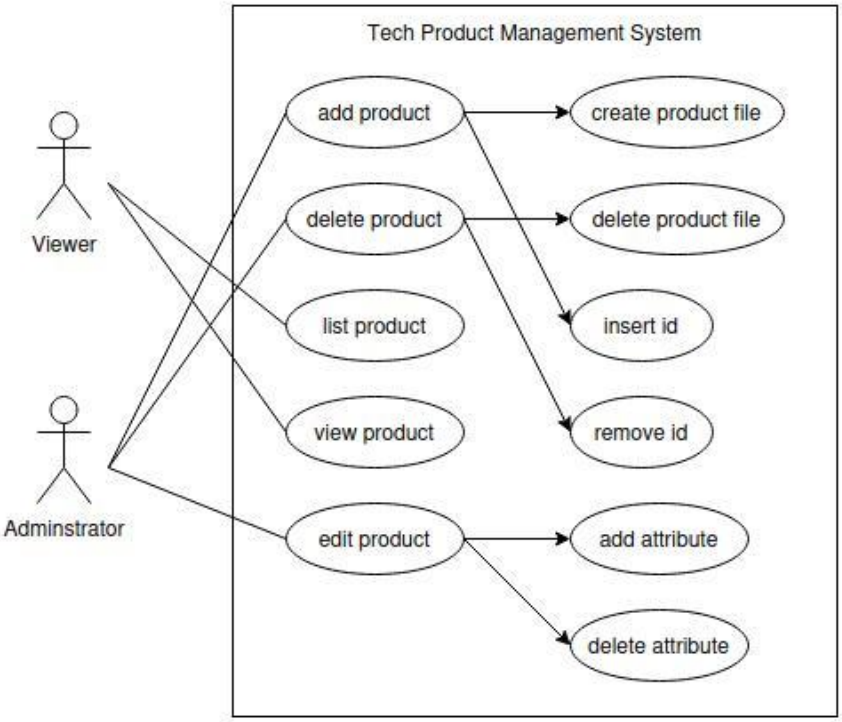
Fig.28 _idlist.txt example *Fig.29 product file example*

Due to considerations, we decide to use a system that read and writes file every time a modification is made. Although not effective in a file handling sense, it will be enough for a simple program with limitations like this.

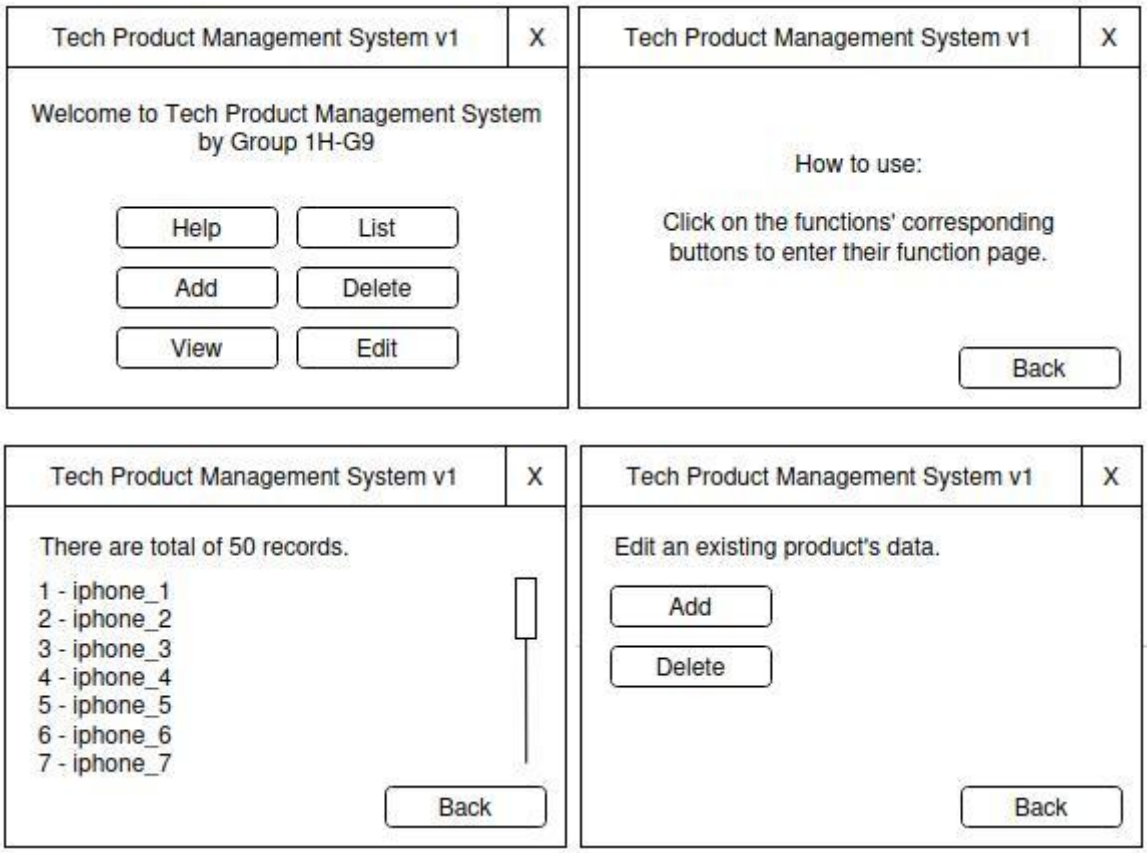
[Fig.28] is an example for the file “_idlist.txt”. Every line in “_idlist.txt” is an individual product id. Every product id must not contain spaces. The contents of “_idlist.txt” should not be manually modified unless you know what you are doing, or it will cause data anomalies or errors.

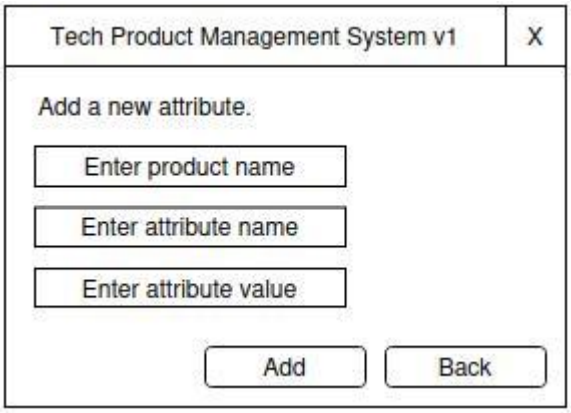
[Fig.29] is an example for a typical product file. Every product id in idlist should have its corresponding product file. Every odd line (eg. 1, 3, 5...) is a “key” and every even line (eg. 2, 4, 6...) is a value related to the key above itself. Every key and value must not contain spaces. The contents of the product data text file should not be manually modified unless you know what you are doing, or it will cause data anomalies or errors.

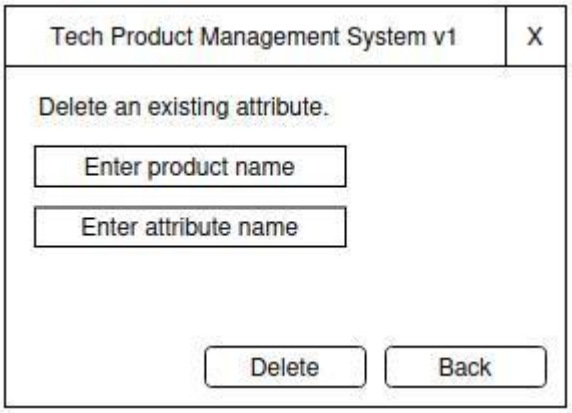
Use-case diagram:

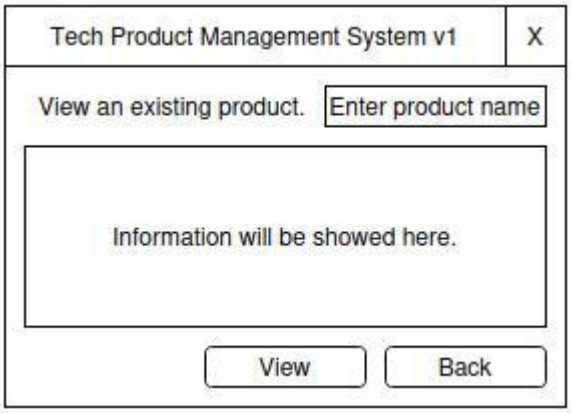


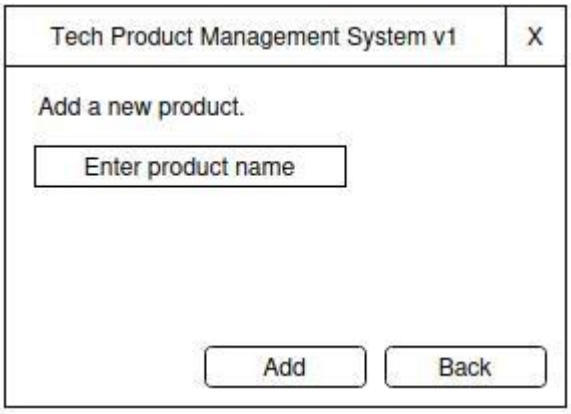
Below are some **draft interface designs**:

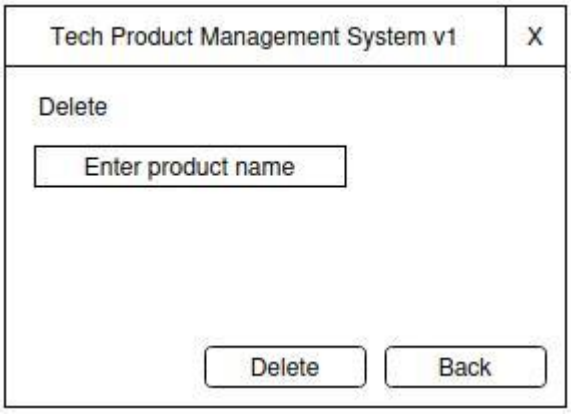



Tech Product Management System v1
X
Add a new attribute.
Enter product name
Enter attribute name
Enter attribute value
Add
Back


Tech Product Management System v1
X
Delete an existing attribute.
Enter product name
Enter attribute name
Delete
Back


Tech Product Management System v1
X
View an existing product.
Enter product name
Information will be showed here.
View
Back


Tech Product Management System v1
X
Add a new product.
Enter product name
Add
Back


Tech Product Management System v1
X
Delete
Enter product name
Delete
Back

Screenshot of the program

Provide screenshots of the program with the explanation. It should be treated as a user manual.

1. Firstly, the program will output a welcome message along with a help menu listing all the available commands in the program. When the user run the program without creating a ***bin*** folder, an error message will be outputted, which tells the user to **create a *bin* folder manually** in the project directory. The program will be terminated.

```
Welcome to Tech Product Management System, by group 1H-G9

--

Below are all available commands:

exit
- End program

help
- Display help

list
- List all product IDs

add <product id>
- Add a new product

view <product id>
- View an existing product

edit <product id> <a/d> <attribute> <value>
- Edit or delete product attribute

del <product id>
- Delete a product

Usage: type <command> into terminal

Stacking inputs, ex.
"add iphone_10"
"view iphone_8 del iphone_9"
"edit iphone_10 a cpu hexacore_2.39GHz"

Values (eg. id, attributes) must not contain spaces!

CRITICAL: Directory "bin/" does not exist, *** manual folder creation is needed ***

Program is terminating
```

2. After the user manually create a ***bin*** folder and run the program, the program will then create a text file called ***_idlist.txt*** under the ***bin*** folder automatically.

```
Welcome to Tech Product Management System, by group 1H-G9

--

Below are all available commands:

exit
- End program

help
- Display help

list
- List all product IDs

add <product id>
- Add a new product

view <product id>
- View an existing product

edit <product id> <a/d> <attribute> <value>
- Edit or delete product attribute

del <product id>
- Delete a product

Usage: type <command> into terminal

Stacking inputs, ex.
"add iphone_10"
"view iphone_8 del iphone_9"
"edit iphone_10 a cpu hexacore_2.39GHz"

Values (eg. id, attributes) must not contain spaces!

WARN: File "bin/_idlist.txt" missing, creating one

---

>
```

help Command

3. If the user enters the ***help*** command, the program will display a help menu listing all the available commands.

```
---

>help
"help" inputted

Below are all available commands:

exit
- End program

help
- Display help

list
- List all product IDs

add <product id>
- Add a new product

view <product id>
- View an existing product

edit <product id> <a/d> <attribute> <value>
- Edit or delete product attribute

del <product id>
- Delete a product

Usage: type <command> into terminal

Stacking inputs, ex.
"add iphone_10"
"view iphone_8 del iphone_9"
"edit iphone_10 a cpu hexacore_2.39GHz"

Values (eg. id, attributes) must not contain spaces!

---

>
```

add Command

If the user enters the add command, the program will prompt them to input a new product ID (which must not contain spaces).

4. Then, the program will check whether the product ID already exists. If it **does not exist**, the product ID will be created, and a text file associated with that product will be generated.

```
---

>add
"add" inputted

Enter new product id: Iphone16
"Iphone16" inputted

Product with id "Iphone16" sucessfully created

---
```

5. If the product ID **already exist**, the program will display a message that tells the user the product ID already exists.

```
---

>add
"add" inputted

Enter new product id: Iphone16
"Iphone16" inputted

Id "Iphone16" already exists

---
```

6. The program also supports **stacked inputs**, allowing multiple commands to be entered in a single line.

```
---

>add Iphone14  add Iphone15  add Iphone16
"add" inputted

Enter new product id: "Iphone14" inputted

Product with id "Iphone14" sucessfully created

---

>"add" inputted

Enter new product id: "Iphone15" inputted

Product with id "Iphone15" sucessfully created

---

>"add" inputted

Enter new product id: "Iphone16" inputted

Id "Iphone16" already exists

---
```

list Command

7. If the user enters the ***list*** command without adding any products to the system, the program will display the message: "**No records available**".

```
---

>list
"list" inputted

No records available

---
```

8. After the user adds products to the system, the ***list*** command will display the total number of records (products) in the system and list them **alphabetically**.

```
---

>add Iphone16  add Asus_Zenbook  add Huawei_Mate70
"add" inputted

Enter new product id: "Iphone16" inputted

Product with id "Iphone16" sucessfully created

---

>"add" inputted

Enter new product id: "Asus_Zenbook" inputted

Product with id "Asus_Zenbook" sucessfully created

---

>"add" inputted

Enter new product id: "Huawei_Mate70" inputted

Product with id "Huawei_Mate70" sucessfully created

---

>list
"list" inputted

Total of 3 records

1       Asus_Zenbook
2       Huawei_Mate70
3       Iphone16

---
```

edit Command

9. If the user enters the ***edit*** command, the system will then prompt the user to enter the product ID that need to be edit:

   If the product ID **does not exist**, an error message will be displayed:

```
---

>edit
"edit" inputted

Enter product id: Iphone15
"Iphone15" inputted

Id "Iphone15" does not exist

---
```

   If the product ID exists, the system will prompt the user to choose between adding or deleting an attribute by entering “a” for add or “d” for delete.

   (i) If **option “a” (add)** is entered, the system will ask the user to input the attribute name for adding. If the attribute does not already exist, the system will create a new attribute and prompt the user to enter the corresponding value for that attribute.

```
---

>edit
"edit" inputted

Enter product id: Iphone16
"Iphone16" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: a
"a" inputted

Enter attribute name: Battery
"Battery" inputted

Enter value: 3561mAh
"3561mAh" inputted

"Battery = 3561mAh" added to product "Iphone16"

---
```

   Else, if the attribute entered already exists, the system will display a message informing the user that the attribute they entered already exists.

```
---

>edit Iphone16
"edit" inputted

Enter product id: "Iphone16" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: a
"a" inputted

Enter attribute name: Battery
"Battery" inputted

Attribute "Battery" already exists

---
```

(ii) If **option "d" (delete)** is entered, the system will ask the user to input the attribute name for delete. If the attribute exists, the system will delete the attribute and its corresponding value.

```
---

>edit Iphone16
"edit" inputted

Enter product id: "Iphone16" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: d
"d" inputted

Enter attribute name: Battery
"Battery" inputted

"Battery" deleted from product "Iphone16"

---
```

Else, if the attribute entered does not already exist, the system will display a message informing the user that the attribute they entered does not exists.

```
---

>edit Iphone16
"edit" inputted

Enter product id: "Iphone16" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: d
"d" inputted

Enter attribute name: Camera
"Camera" inputted

Attribute "Camera" does not exist

---
```

(iii) If the user input an **invalid option other than "a" and "d"**, the system will display an error message that informs the user the option is invalid.

```
---

>edit Iphone16
"edit" inputted

Enter product id: "Iphone16" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: s
"s" inputted

Invalid option: "s"

---
```

view Command

10. If the user enters the ***view*** command, the system will then prompt the user to enter the product ID that need to be view:

    If the product ID **does not exist**, an error message will be displayed:

```
---

>view
"view" inputted

Enter product id: Huawei_Mate60
"Huawei_Mate60" inputted

Id "Huawei_Mate60" does not exist

---
```

(i) If the product ID exists, but there is no information is available for the product, the system will displayed a message: “No information available”.

```
---

>view Huawei_Mate70
"view" inputted

Enter product id: "Huawei_Mate70" inputted

Viewing product "Huawei_Mate70" information:

No information available

---
```

(ii) If the product exists and has associated attribute(s) and value(s), the system will display all the product information, with the attributes listed alphabetically.

```
---

>edit Huawei_Mate70 a OS HarmonyOS_4.3
"edit" inputted

Enter product id: "Huawei_Mate70" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: "a" inputted

Enter attribute name: "OS" inputted

Enter value: "HarmonyOS_4.3" inputted

"OS = HarmonyOS_4.3" added to product "Huawei_Mate70"

---

>edit Huawei_Mate70 a Battery 5300mAh
"edit" inputted

Enter product id: "Huawei_Mate70" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: "a" inputted

Enter attribute name: "Battery" inputted

Enter value: "5300mAh" inputted

"Battery = 5300mAh" added to product "Huawei_Mate70"

---

>view Huawei_Mate70
"view" inputted

Enter product id: "Huawei_Mate70" inputted

Viewing product "Huawei_Mate70" information:

Battery = 5300mAh
OS = HarmonyOS_4.3

---
```

del Command

11. If the user enters the ***del*** (delete) command, the system will then prompt the user to enter the product ID that need to be delete:

    (i) If the product ID **does not exist**, an error message will be displayed:

```
---

>del
"del" inputted

Enter product id: Iphone14
"Iphone14" inputted

Id "Iphone14" does not exist

---
```

    (ii) If the product ID **exists**, the product ID will be deleted, and its record in the ***_idlist.txt*** and its associated text file ***<productID>.txt*** will be deleted, and lastly the system will output a message that the product has successfully deleted.

```
---

>del
"del" inputted

Enter product id: Asus_Zenbook
"Asus_Zenbook" inputted

Product "Asus_Zenbook" successfully deleted

---
```

exit Command

12. If the user enters the ***exit*** command, the system will be terminated.

```
---

>exit
"exit" inputted

Program is terminating
```

Invalid Command

13. If the user enters an invalid command, an error message will be displayed:

```
---

>delete
"delete" inputted

Invalid command: "delete"

---
```

Example of Stacked Input

Our system also supports stacked input. An example input is provided below:

```
---

>add Iphone16_ProMax edit Iphone16_ProMax a Chip A18_Pro edit Iphone16_ProMax a OS iOS_18 edit Iphone16_ProMax a Battery 4685mAh view Iphone16_ProMax
"add" inputted

Enter new product id: "Iphone16_ProMax" inputted

Product with id "Iphone16_ProMax" sucessfully created

---

>"edit" inputted

Enter product id: "Iphone16_ProMax" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: "a" inputted

Enter attribute name: "Chip" inputted

Enter value: "A18_Pro" inputted

"Chip = A18_Pro" added to product "Iphone16_ProMax"

---

>"edit" inputted

Enter product id: "Iphone16_ProMax" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: "a" inputted

Enter attribute name: "OS" inputted

Enter value: "iOS_18" inputted

"OS = iOS_18" added to product "Iphone16_ProMax"

---

>"edit" inputted

Enter product id: "Iphone16_ProMax" inputted

Choose to enter "a" (add attribute) or "d" (delete attribute)

Option: "a" inputted

Enter attribute name: "Battery" inputted

Enter value: "4685mAh" inputted

"Battery = 4685mAh" added to product "Iphone16_ProMax"

---

>"view" inputted

Enter product id: "Iphone16_ProMax" inputted

Viewing product "Iphone16_ProMax" information:

Battery = 4685mAh
Chip = A18_Pro
OS = iOS_18

---
```

Source Code

Please provide the full C++ source code for your project. Ensure that the code includes clear, descriptive comments to help readers understand its functionality and logic.

Note: The source code submission will be used for plagiarism detection and verification of AI-generated content.

```
#include <iostream>
#include <fstream> //file io library
using namespace std;


//consts

const string BIN_PATH = "bin/"; //directory to store idlist.txt and product files


//class definitions

class Idlist { //class for idlist

    const string LIST_PATH = BIN_PATH + "_idlist.txt"; //_prefix so file always on
top

public:

    string getPath() {return LIST_PATH;}

    int len() { //return idlist length (id count)
        int n = 0;
        string line;
        ifstream ifile(LIST_PATH);
        while (getline(ifile, line)) n++;
        return n;
    }

    bool exist(string id) { //check if id is in idlist
        string line;
        ifstream ifile(LIST_PATH);
        while (getline(ifile, line)) {
            if (line == id) return true;
        }
        return false;
    }

    void listAll() { //print all items in idlist
        int idx = 1;
        string line;
        ifstream ifile(LIST_PATH);
        while (getline(ifile, line)) {
            cout << idx++ << "\t" << line << "\n";
        }
    }
```

```
    void insertId(string id) { //insert new id into idlist
        int idx = 0, nlen = len() + 1;
        string lines[nlen], line;
        ifstream ifile(LIST_PATH);
        //add items until line is smaller than id or EOF
        while (getline(ifile, line) && line < id) lines[idx++] = line;
        //insert new id into sorted array
        lines[idx++] = id;
        //add remaining items if not EOF
        if (idx < nlen) {
            lines[idx++] = line;
            while (getline(ifile, line)) lines[idx++] = line;
        }
        //write to idlist
        ofstream ofile(LIST_PATH);
        for (string a : lines) ofile << a << "\n";
        ofile.close();
    }

    void deleteId(string id) { //delete id from idlist
        int idx = 0, nlen = len() - 1;
        string lines[nlen], line;
        ifstream ifile(LIST_PATH);
        //add items until line is id or EOF
        while (getline(ifile, line) && line != id) lines[idx++] = line;
        //ignore (discard) line, add remaining if not EOF
        if (idx < nlen) {
            while (getline(ifile, line)) lines[idx++] = line;
        }
        //write to idlist
        ofstream ofile(LIST_PATH);
        for (string a : lines) ofile << a << "\n";
        ofile.close();
    }

} idlist; //idlist object


class Product { //class for products

    string id, path; //identifier, product file path

public:

    string getId() {return id;} //get id

    void setId(string nid) { //set id and path
        id = nid;
        path = BIN_PATH + id + ".txt";
    }

    void create() { //create product
        idlist.insertId(id);
        ofstream ofile(path);
        ofile.close();
    }

    void del() { //delete product
```

```
        idlist.deleteId(id);
        remove(path.c_str());
    }

    int len() { //get product file length
        int n = 0;
        string line;
        ifstream ifile(path);
        while (getline(ifile, line)) n++;
        return n;
    }

    bool existAttr(string attr) { //check if attribute exists
        string line;
        ifstream ifile(path);
        while (getline(ifile, line)) {
            if (line == attr) return true;
            getline(ifile, line);
        }
        return false;
    }

    void addAttr(string attr, string val) { //add attribute
        int idx = 0, nlen = len() + 2;
        string lines[nlen], line;
        ifstream ifile(path);
        //iterate until line smaller than attribute or EOF
        while (getline(ifile, line) && line < attr) {
            lines[idx++] = line;
            getline(ifile, line);
            lines[idx++] = line;
        }
        //add atribute and value to array
        lines[idx++] = attr;
        lines[idx++] = val;
        //add remaining
        if (idx < nlen) {
            lines[idx++] = line;
            while (getline(ifile, line)) {
                lines[idx++] = line;
            }
        }
        //write to product file
        ofstream ofile(path);
        for (string a : lines) ofile << a << "\n";
        ofile.close();
    }

    void delAttr(string attr) { //delete attribute
        int idx = 0, nlen = len() - 2;
        string lines[nlen], line;
        ifstream ifile(path);
        //iterate until line smaller than attribute or EOF
        while (getline(ifile, line) && line != attr) {
            lines[idx++] = line;
            getline(ifile, line);
            lines[idx++] = line;
        }
        //discard and add remaining
```

```
        if (idx < nlen) {
            getline(ifile, line);
            while (getline(ifile, line)) {
                lines[idx++] = line;
            }
        }
        //write to product file
        ofstream ofile(path);
        for (string a : lines) ofile << a << "\n";
        ofile.close();
    }

    void view() { //print product info
        string line;
        ifstream ifile(path);
        while (getline(ifile, line)) {
            cout << line << " = ";
            getline(ifile, line);
            cout << line << "\n";
        }
    }

};


//helper functions

bool chkmkDir() { //check dir and make required files
    string LIST_PATH = idlist.getPath();
    ifstream ifile(LIST_PATH);
    if(!ifile) {
        ofstream ofile(LIST_PATH);
        ofile.close();
        ifile.close();
        ifile.open(LIST_PATH);
        //if bin/ doesnt exist, kill input loop and end program
        if(!ifile) {
            cout << "CRITICAL: Directory \"" << BIN_PATH << "\" does not exist,
*** manual folder creation is needed ***\n\n";
            return false;
        }
        cout << "WARN: File \"" << LIST_PATH << "\" missing, creating one\n\n";
    }
    ifile.close();
    return true;
}


string getInput(string prompt) { //formatted input prompt
    string ans;
    cout << prompt;
    cin >> ans;
    cout << "\"" << ans << "\" inputted\n\n";
    return ans;
}


//command functions
```

```
void cmdHelp() { //help command
    //display help message
    cout <<
    "Below are all available commands:\n\n" <<
    "exit \n- End program\n\n" <<
    "help \n- Display help\n\n" <<
    "list \n- List all product IDs\n\n" <<
    "add <product id> \n- Add a new product\n\n" <<
    "view <product id> \n- View an existing product\n\n" <<
    "edit <product id> <a/d> <attribute> <value> \n- Edit or delete product
attribute\n\n" <<
    "del <product id> \n- Delete a product\n\n" <<
    "Usage: type <command> into terminal\n\n" <<
    "Stacking inputs, ex.\n" <<
    "\"add iphone_10\"\n" <<
    "\"view iphone_8 del iphone_9\"\n" <<
    "\"edit iphone_10 a cpu hexacore_2.39GHz\"\n\n" <<
    "Values (eg. id, attributes) must not contain spaces!\n\n";
}


void cmdAdd() { //add command
    //get product id
    Product prd;
    prd.setId(getInput("Enter new product id: "));
    //check if id exist
    if (idlist.exist(prd.getId())) {
        cout << "Id \"" << prd.getId() << "\" already exists\n\n";
        return;
    }
    //create product entry
    prd.create();
    cout << "Product with id \"" << prd.getId() << "\" sucessfully created\n\n";
}


void cmdList() { //list command
    //list items with index if there is any
    int nlen = idlist.len();
    if (nlen > 0) {
        cout << "Total of " << nlen << " records\n\n";
        idlist.listAll();
        cout << "\n";
    } else {
        cout << "No records available\n\n";
    }
}


void cmdView() { //view command
    //get product id
    Product prd;
    prd.setId(getInput("Enter product id: "));
    //check if id exists
    if (!idlist.exist(prd.getId())) {
        cout << "Id \"" << prd.getId() << "\" does not exist\n\n";
        return;
    }
    //view product info
```

```
    cout << "Viewing product \"" << prd.getId() << "\" information:\n\n";
    if (prd.len() == 0) {
        cout << "No information available\n";
    } else {
        prd.view();
    }
    cout << "\n";
}


void cmdEdit() { //edit command
    //get product id
    Product prd;
    prd.setId(getInput("Enter product id: "));
    //check if id exists
    if (!idlist.exist(prd.getId())) {
        cout << "Id \"" << prd.getId() << "\" does not exist\n\n";
        return;
    }
    //ask add or delete attribute
    cout << "Choose to enter \"a\" (add attribute) or \"d\" (delete
attribute)\n\n";
    string opt = getInput("Option: ");
    if (opt == "a") {
        //get and check attribute name
        string attr = getInput("Enter attribute name: ");
        if (prd.existAttr(attr)) {
            cout << "Attribute \"" << attr << "\" already exists\n\n";
            return;
        }
        //get value
        string val = getInput("Enter value: ");
        //add attribute and value
        prd.addAttr(attr, val);
        cout << "\"" << attr << " = " << val << "\" added to product \"" <<
prd.getId() << "\"\n\n";
    } else if (opt == "d") {
        //get and check attribute name
        string attr = getInput("Enter attribute name: ");
        if (!prd.existAttr(attr)) {
            cout << "Attribute \"" << attr << "\" does not exist\n\n";
            return;
        }
        //delete attribute
        prd.delAttr(attr);
        cout << "\"" << attr  << "\" deleted from product \"" << prd.getId() <<
"\"\n\n";
    } else {
        cout << "Invalid option: \"" << opt << "\"\n\n";
    }
}


void cmdDel() {
    //get product id
    Product prd;
    prd.setId(getInput("Enter product id: "));
    //check if id exists
    if (!idlist.exist(prd.getId())) {
```

```
        cout << "Id \"" << prd.getId() << "\" does not exist\n\n";
        return;
    }
    //delete product
    prd.del();
    cout << "Product \"" << prd.getId() << "\" successfully deleted\n\n";
}


//input loop function
bool inputCmd() {
    //input, use normal cin
    cout << "---\n\n"; //separator
    string cmd = getInput(">");
    //process input
    if (cmd == "exit")          return false; //kill input loop
    else if (cmd == "help")     cmdHelp();
    else if (cmd == "add")      cmdAdd();
    else if (cmd == "list")     cmdList();
    else if (cmd == "view")     cmdView();
    else if (cmd == "edit")     cmdEdit();
    else if (cmd == "del")      cmdDel();
    else cout << "Invalid command: \"" << cmd << "\"\n\n";
    //keep input loop alive
    return true;
}


//main
int main() {
    //inits
    cout <<
    "\nWelcome to Tech Product Management System, by group 1H-G9\n\n" <<
    "--\n\n";
    cmdHelp();
    //check required file and dir
    bool alive = chkmkDir();
    //input loop
    while (alive && cin) {
        alive = inputCmd();
    }
    //exit
    cout << "Program is terminating\n";
}
```

Bibliography

[1] "Standard library header - cppreference.com," *Cppreference.com*, 2024. https://en.cppreference.com/w/cpp/header/fstream

[2] GeeksforGeeks, "Object Oriented Programming in C++ - GeeksforGeeks," *GeeksforGeeks*, Aug. 08, 2015. https://www.geeksforgeeks.org/object-oriented-programming-in-cpp/

[3] GeeksforGeeks, "Insertion Sort Algorithm," *GeeksforGeeks*, Mar. 07, 2013. https://www.geeksforgeeks.org/insertion-sort-algorithm/

[4] N. Gunnoo, "Manual Data Entry: Challenges and solutions in 2023 | Parseur," *parseur.com*, Jun. 18, 2022. https://parseur.com/blog/manual-data-entry

[5] E. Kaminski, "Spreadsheet Errors: The Hidden Risk In MedTech," *Forbes*, Jan. 28, 2025. Available: https://www.forbes.com/councils/forbestechcouncil/2025/01/28/spreadsheet-errors-the-hidden-risk-in-medtech/

[6] "Records Storage Standards Toolkit," *National Archives*, Aug. 15, 2016. https://www.archives.gov/records-mgmt/storage-standards-toolkit

[7] N. Fearn, "Best document management software of 2019: DMS systems for managing files and more," *TechRadar*, Aug. 16, 2019. https://www.techradar.com/best/best-document-management-software

[8] M. Breed, "The Hidden Costs of Data Entry Mistakes | The Warren Group," *The Warren Group*, Aug. 2024, doi: https://doi.org/1021426/et-core-unified-1021426.

[9] Nashmil Mobasseri, "3 Key Principles for Creating an Intuitive User Interface," *Medium*, Sep. 26, 2023. https://medium.com/design-bootcamp/3-key-principles-for-creating-an-intuitive-user-interface-6189a6165134

[10] J. Fruhlinger, "What is information security? Definition, principles, and jobs," *CSO Online*, Jan. 17, 2020. https://www.csoonline.com/article/568841/what-is-information-security-definition-principles-and-jobs.html

Appendix

Attach some supporting documents or information that are not included in the previous sections.

Presentation video link:

https://mmuedumy-my.sharepoint.com/:v:/g/personal/cheong_kai_jin_student_mmu_edu_my/Eak-a1mgF7tPhAyIOWP1ifgB_dEFc6O77zL_RG87pf8Wyg?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZXJyYWxBcHBQbGF0Zm9ybSI6IldlYiIsInJlZmVycmFsTW9kZSI6InZpZXciLCJyZWZlcnJhbFZpZXciOiJNeUZpbGVzTGlua0NvcHkifX0&e=ngljDP

Presentation canva link:

https://www.canva.com/design/DAGePDlw1S8/xuFPY6xPIuhCH82-DERC1A/edit?utm_content=DAGePDlw1S8&utm_campaign=designshare&utm_medium=link2&utm_source=sharebutton